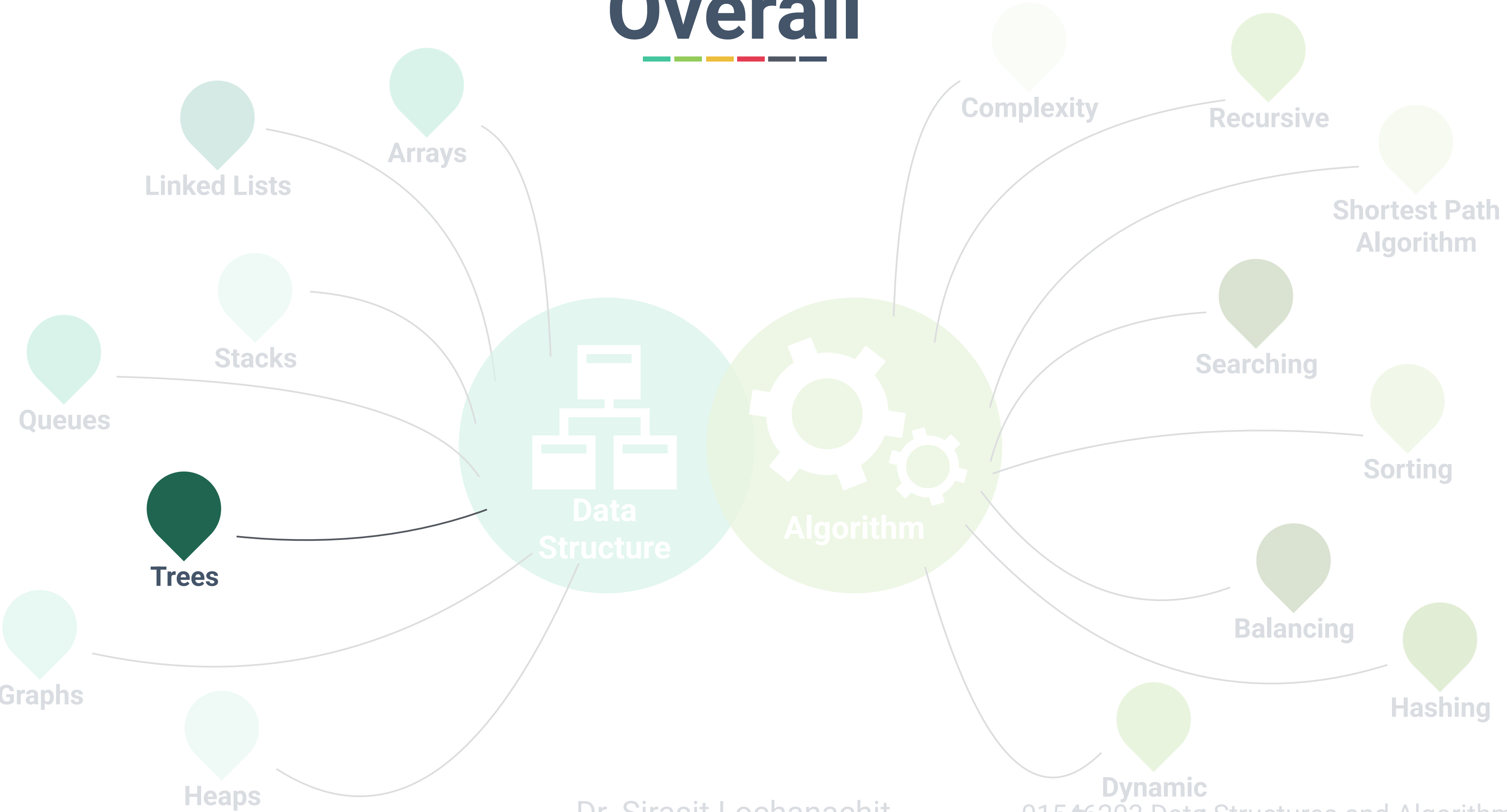
Week 6: Trees

Dr. Sirasit Lochanachit

Overall


Arrays
Linked Lists
Stacks
Queues
Trees
Graphs
Heaps
Data Structure
Algorithm
Complexity
Recursive
Shortest Path Algorithm
Searching
Sorting
Balancing
Hashing
Dynamic Programming

Abstract Data Type

Linear:

- Arrays, Stacks, Queues, and Linked Lists
- Operations: push, pop, enqueue, dequeue
- Algorithms: Searching and sorting, insertion, deletion

Non-Linear:

- Trees and Graphs
- Algorithms: Traversal, Insertion and Deletion, Balancing, and etc.

Today's Outline

General Trees:

- Definition and examples
- Elements of Tree Structure

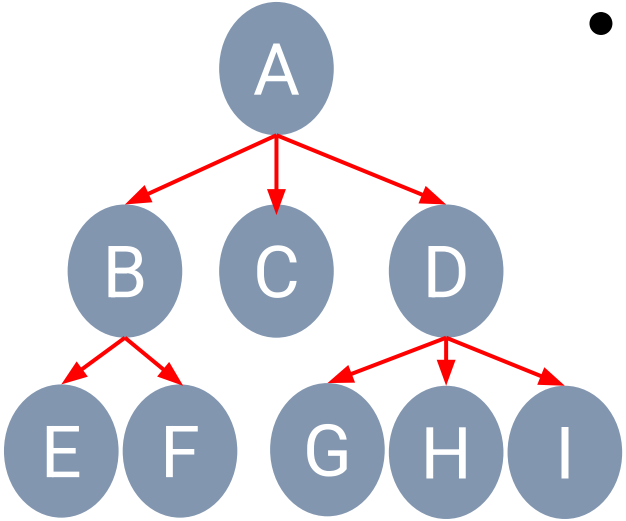
Binary Trees:

- Definition, properties, and types

Tree traversal algorithms:

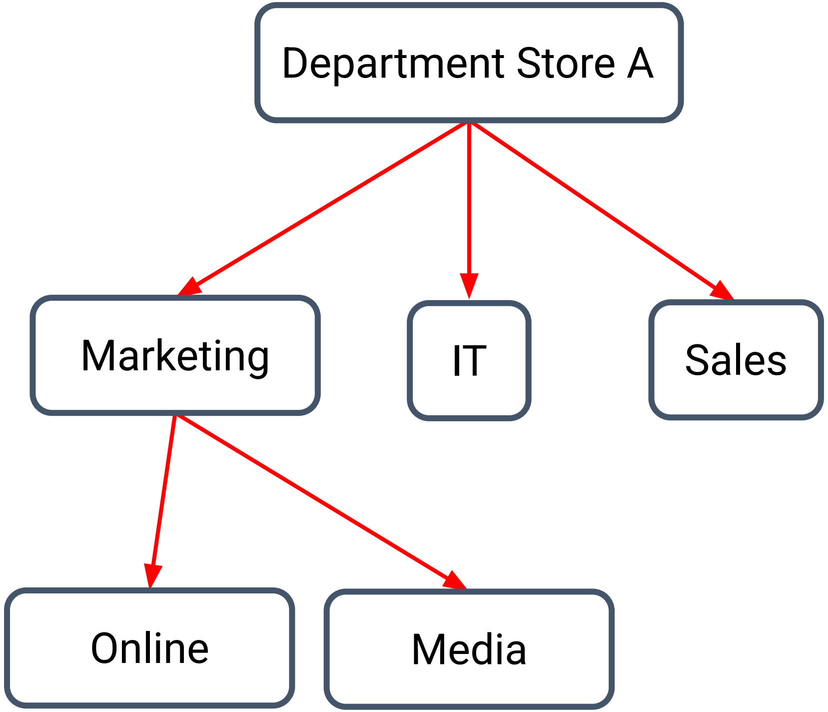
- Depth-first Traversal (Preorder, postorder, and inorder)
- Breadth-first Traversal

What is a Tree?



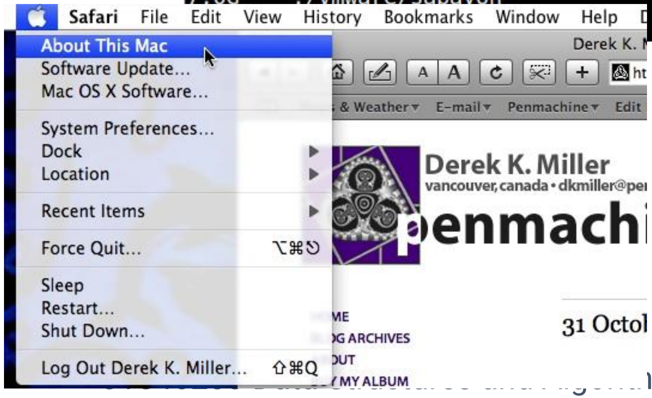
- A **tree** stores elements in a hierarchical structure.

Tree Example

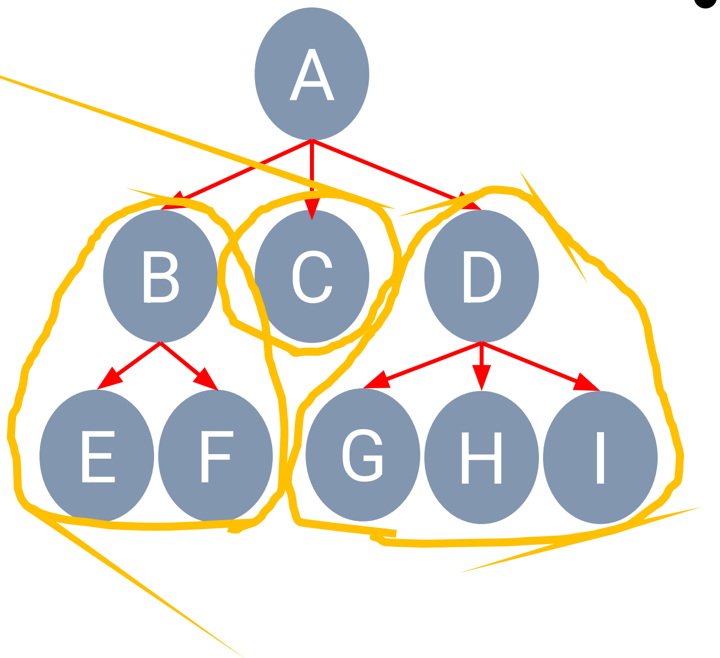


Tree Applications

- Tree represents natural organisation for data.
- Tree structure has been used widely in
  - File systems (Directory)
  - Graphical User Interfaces
  - Databases (Sub-categories, etc.)
  - Websites



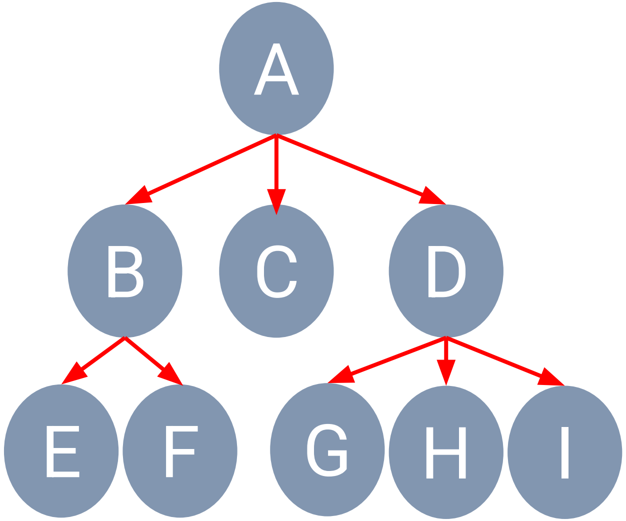
Formal Definition



- A **tree** is a collection of nodes that store elements with a **parent-child** relationship.

[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

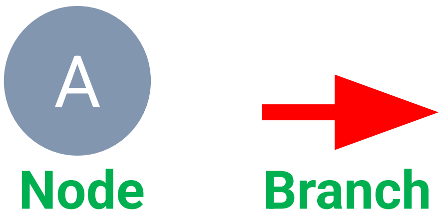
Formal Definition

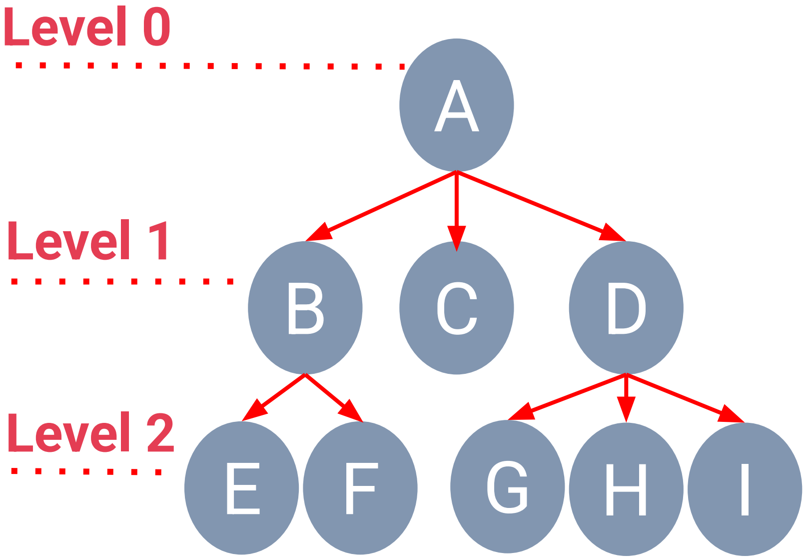


- **Root**
- **Edge**
- **Child**
- **Siblings**
- **Internal** or **<u>parent</u>** node
- **External** or **<u>leaf</u>** node
- **Ancestor**
- **Descendant**

[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

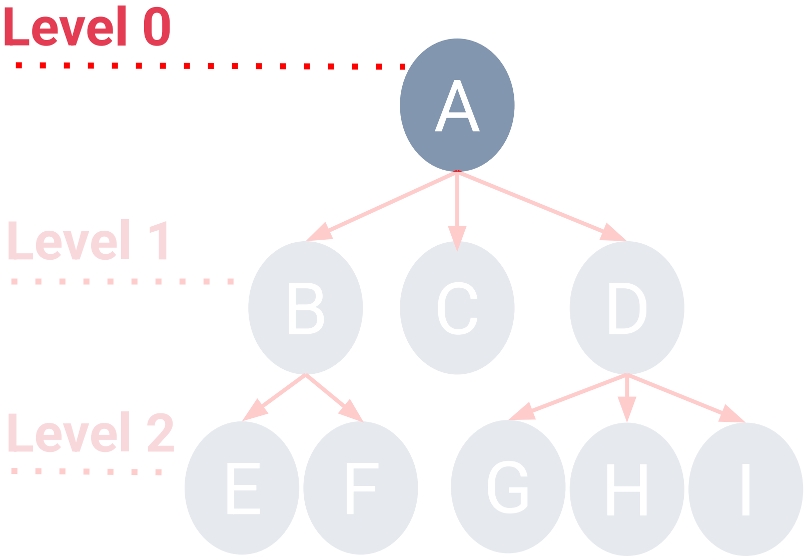
Basic Elements of Tree Structure





Basic Elements of Tree Structure

Root node is the *top element* of a tree.

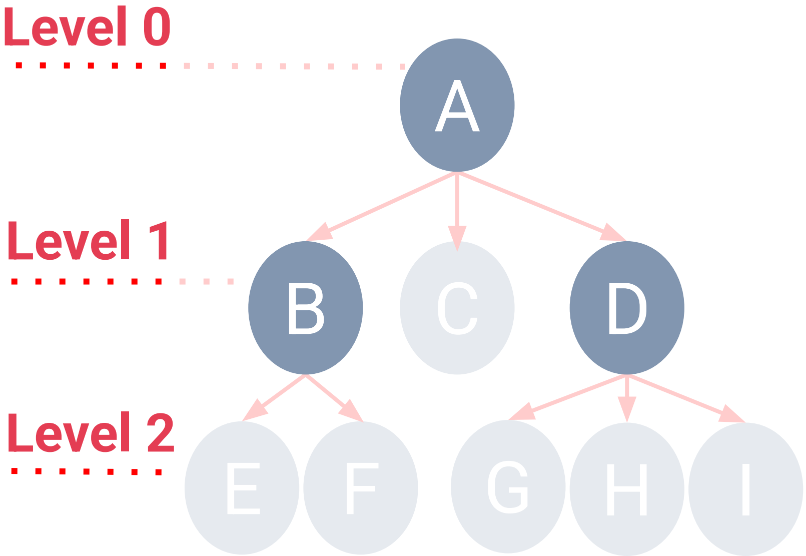


- ***Root*** Node: *A*

Basic Elements of Tree Structure

Parent node is an <u>internal</u> node that has one or more children nodes.

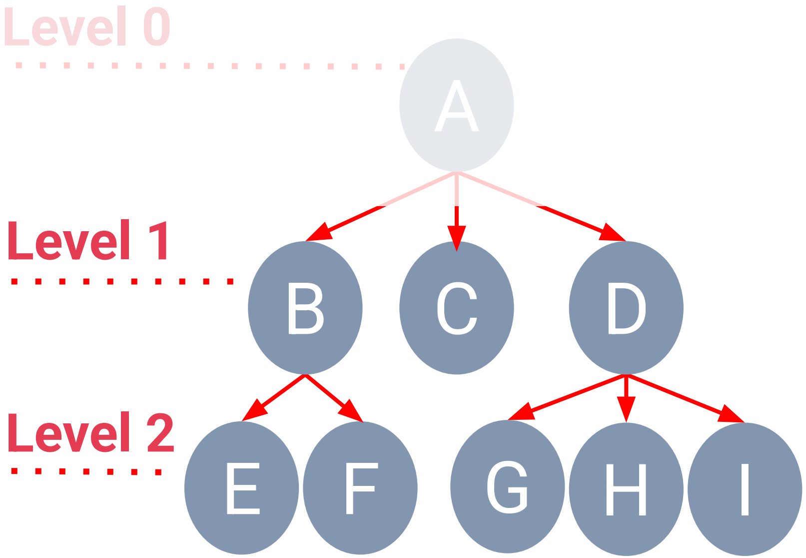
- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*



Basic Elements of Tree Structure

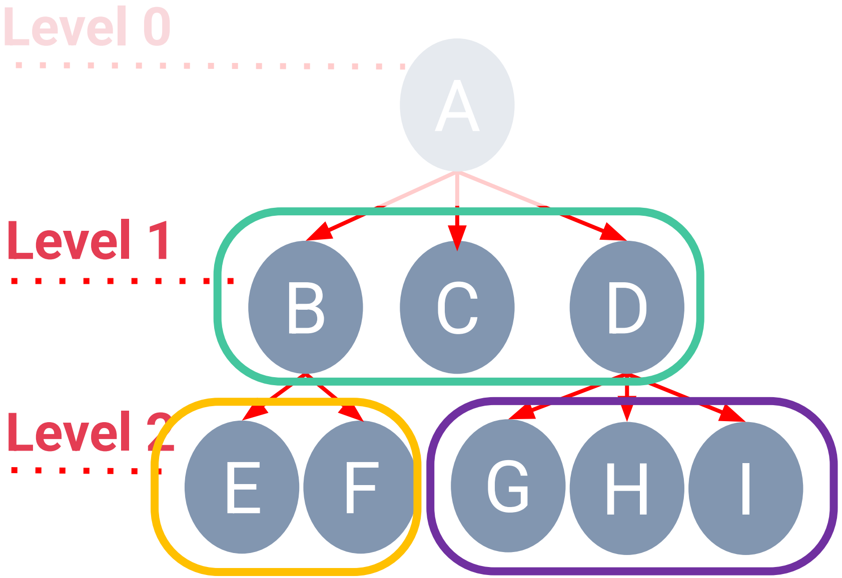
Children node is a node that has parent node.

- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*



Basic Elements of Tree Structure

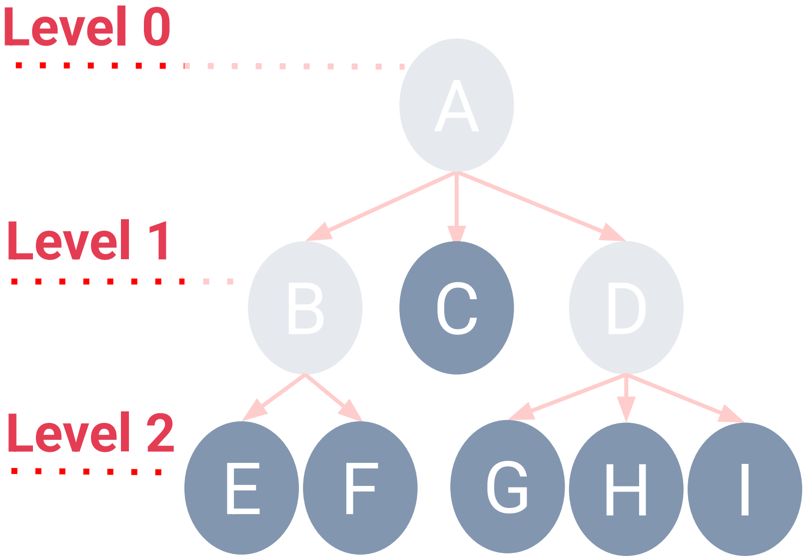
Two or more children nodes are ***siblings*** when they have the same parent node.



- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}

Basic Elements of Tree Structure

Leaf **node is a node without any children.**

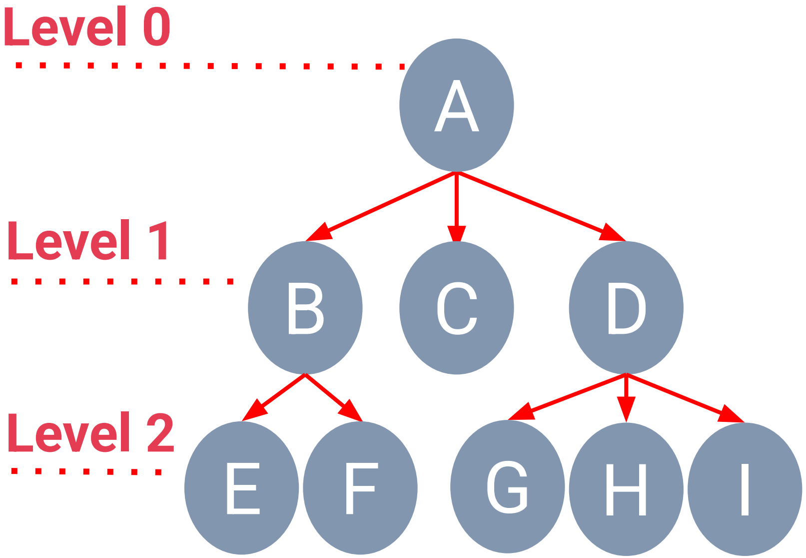


- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- ***Leaf*** Nodes: *C*, *E*, *F*, *G*, *H* and *I*

Basic Elements of Tree Structure

A ***Degree of tree*** is a number of subtrees or edges/links.

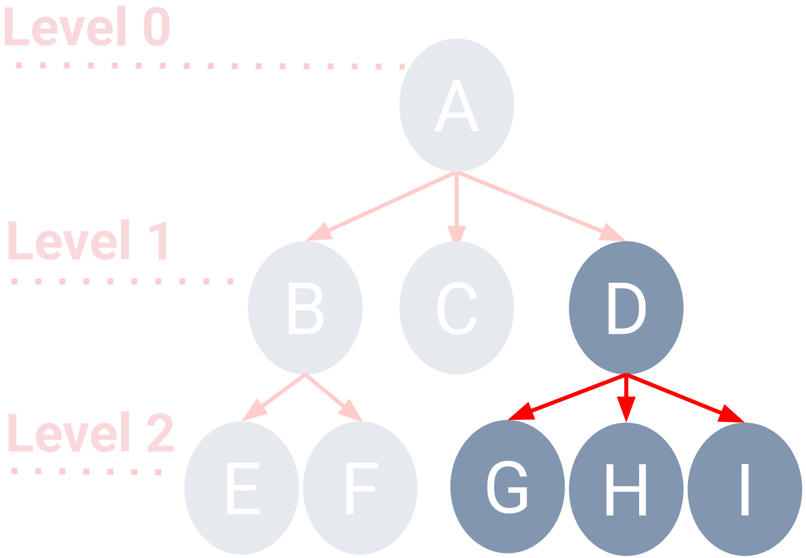
Edge is a pair between parent and child node.



- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- ***Leaf*** Nodes: *C*, *E*, *F*, *G*, *H* and *I*
- Degree of Tree is 8

Basic Elements of Tree Structure

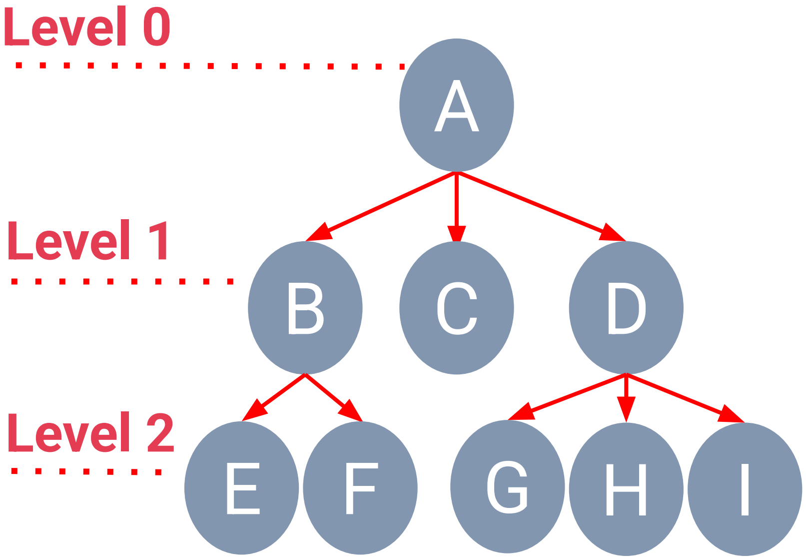
A ***Degree of node*** is a number of edges connected to the node.



- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- ***Leaf*** Nodes: *C*, *E*, *F*, *G*, *H* and *I*
- Degree of Tree is 8
- Degree of Node D is 3

Basic Elements of Tree Structure

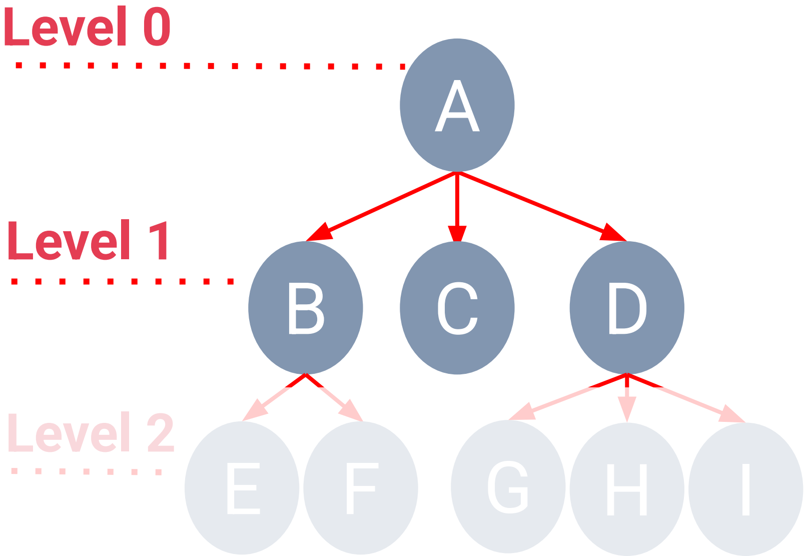
Height of tree is a number of levels starting from the root node.



- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- ***Leaf*** Nodes: *C*, *E*, *F*, *G*, *H* and *I*
- Degree of Tree is 8
- Degree of Node D is 3
- Height of tree is 3

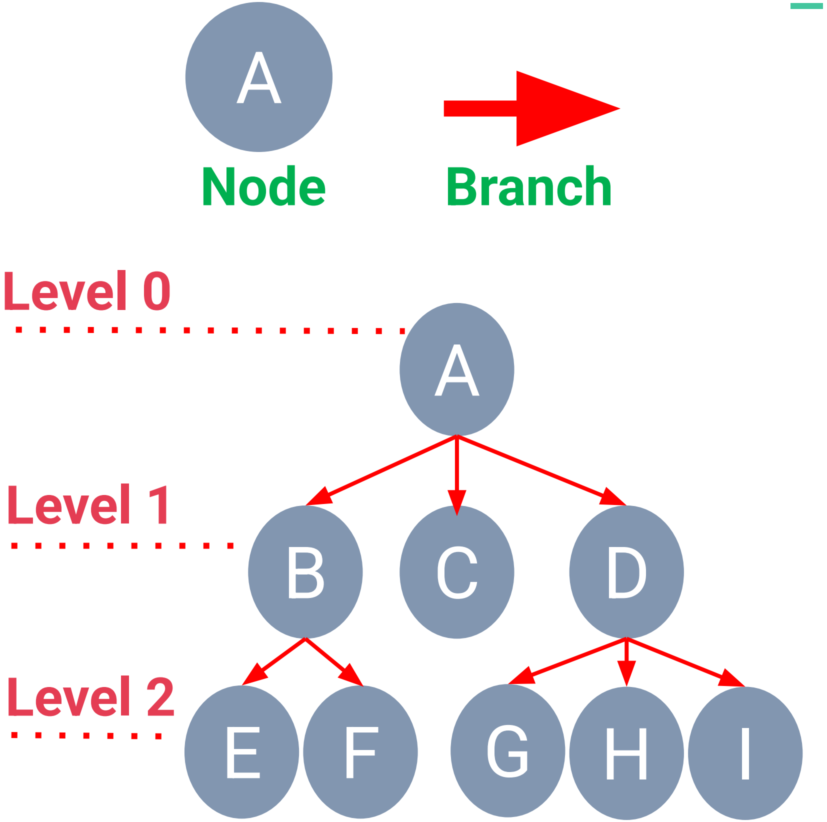
Basic Elements of Tree Structure

Depth of a node is a length of the path to its root.



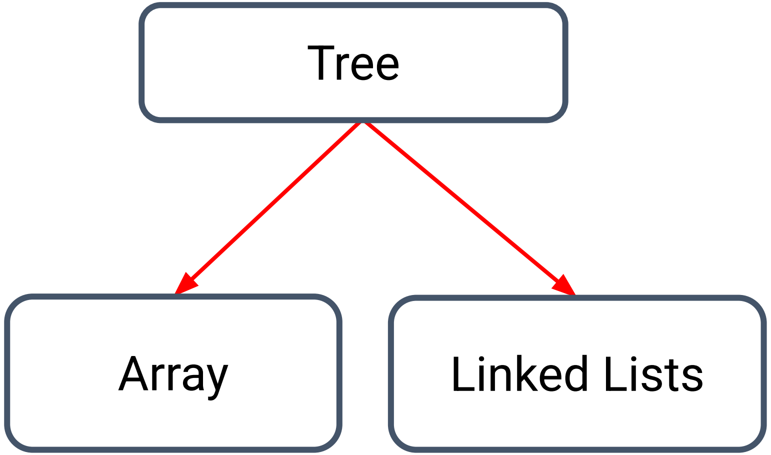
- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- ***Leaf*** Nodes: *C*, *E*, *F*, *G*, *H* and *I*
- Degree of Tree is 8
- Degree of Node D is 3
- Height of tree is 3
- Depth of node D is 1

Basic Elements of Tree Structure

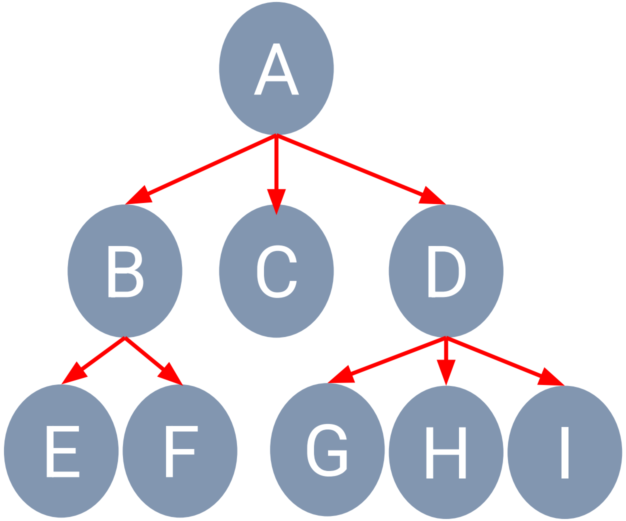


- ***Root*** Node: *A*
- ***Parents***: *A*, *B*, and *D*
- ***Children***: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- ***Siblings***: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- ***Leaf*** Nodes: *C*, *E*, *F*, *G*, *H* and *I*
- Degree of Tree is 8
- Degree of Node D is 3
- Height of tree is 3
- Depth of node D is 1

Implementation of Trees



Other Implementation of Trees

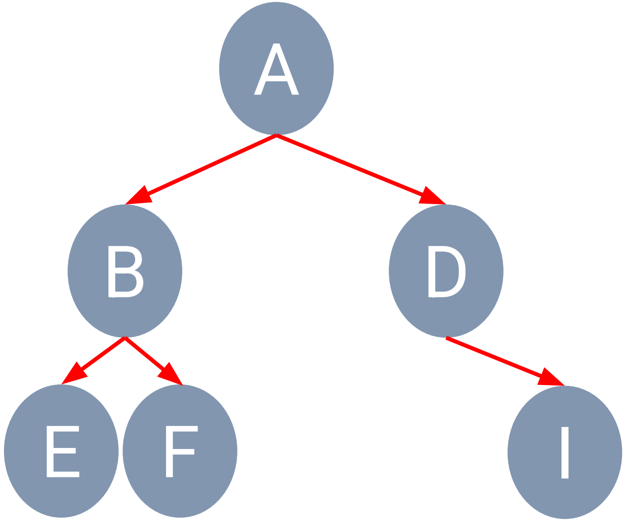


A{ B{E, F}, C, D{G, H, I} }

Linked List Form

Set/List Form

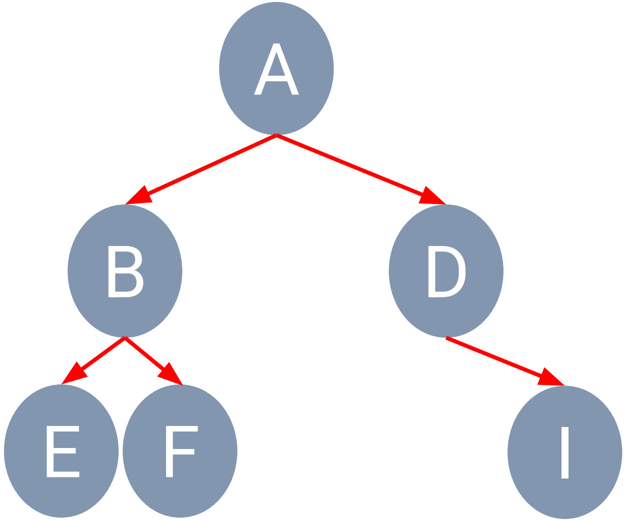
Binary Trees



- A **binary tree** is a tree in which any node can have only two children at maximum.
- Each child node is designated as being either a **left child** or **right child**.

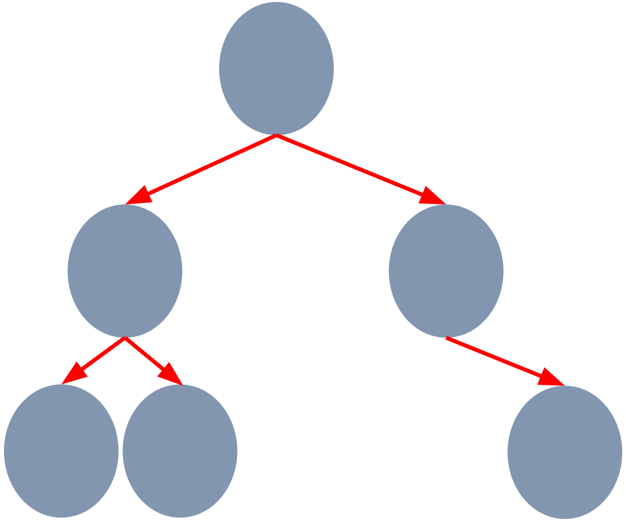
[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

Recursive Binary Trees



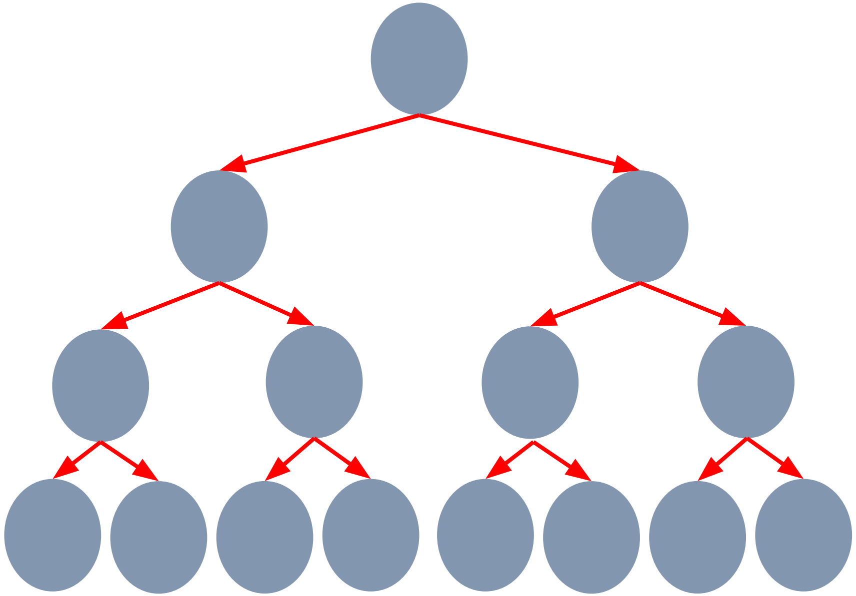
- A **binary tree** is either empty or consists of:
  - A root node that stores an element
  - A binary left subtree (possibly empty)
  - A binary right subtree (possibly empty)

Binary Trees' Properties



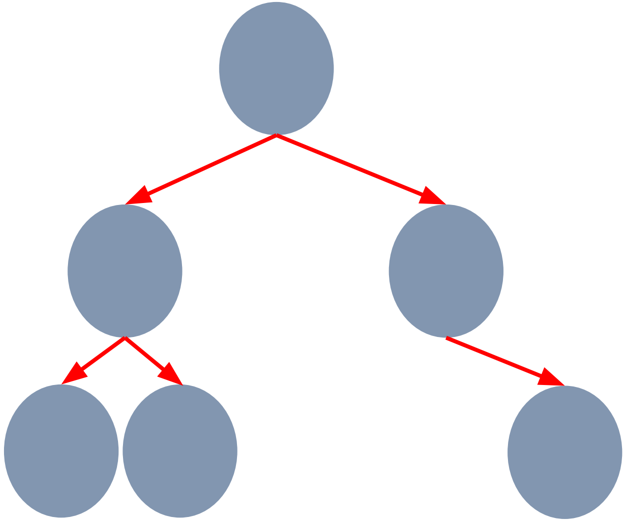
- A **Degree** of node is a number of edges/subtrees connected to the node.
- A **Depth** of a node is a length of the path to its root (i.e. number of edges from root the node).
    - d denotes the depth/level of a node
- **Height** of tree is a number of levels starting from the root node (i.e. number of edges from the node to the deepest leaf).
    - *h* denotes the height of tree

Binary Trees’ Properties



		Number of Nodes In each leavel
Level 0		**1**
Level 1		**2**
Level 2		**4**
Level 3		**8**
Level d		**2^d**

Binary Trees' Properties



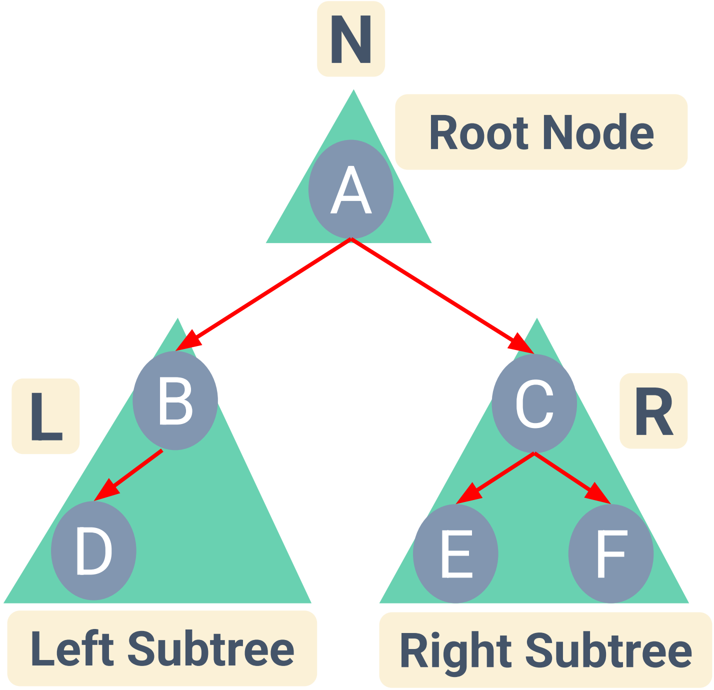
Suppose n is the total number of nodes in a tree and h is the number of height of tree.

- **Maximum height** of binary trees, h_{max} =
- **Minimum height** of binary trees, h_{min} =
- **Minimum number** of nodes in a tree, n_{min} =
- **Maximum number** of nodes in a tree, n_{max} =

Binary Tree Traversal

- A **traversal** of a tree is a method to access all the tree nodes.
- Two main approaches: Depth-first and Breadth-first
  - Depth-first
    - Preorder traversal
    - Postorder traversal
    - Inorder traversal
  - Breadth-first - visit all the nodes at depth *d* before moving to depth *d*+1.

Depth-first Traversal

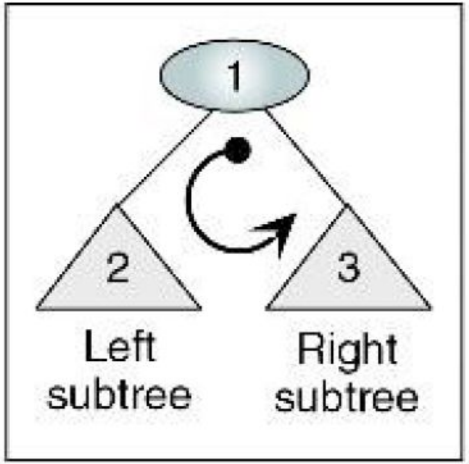


- **Pre**order Traversal
  N L R
- **In**order Traversal
  L N R
- **Post**order Traversal
  L R N

Preorder Traversal

- Root -> Left subtree -> Right subtree

```
Algorithm preOrder(root):
    if (root is not null):
        process(root)
        preOrder(leftSubtree)
        preOrder(rightSubtree)
    end if
end preOrder
```

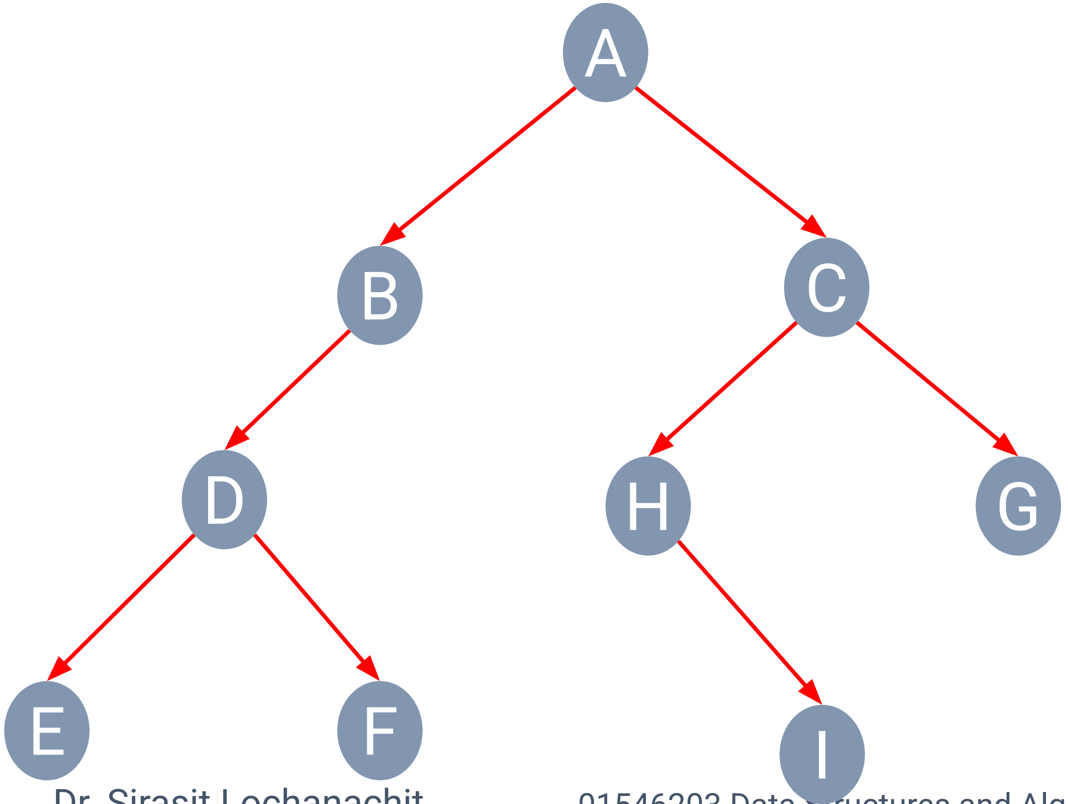


Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**



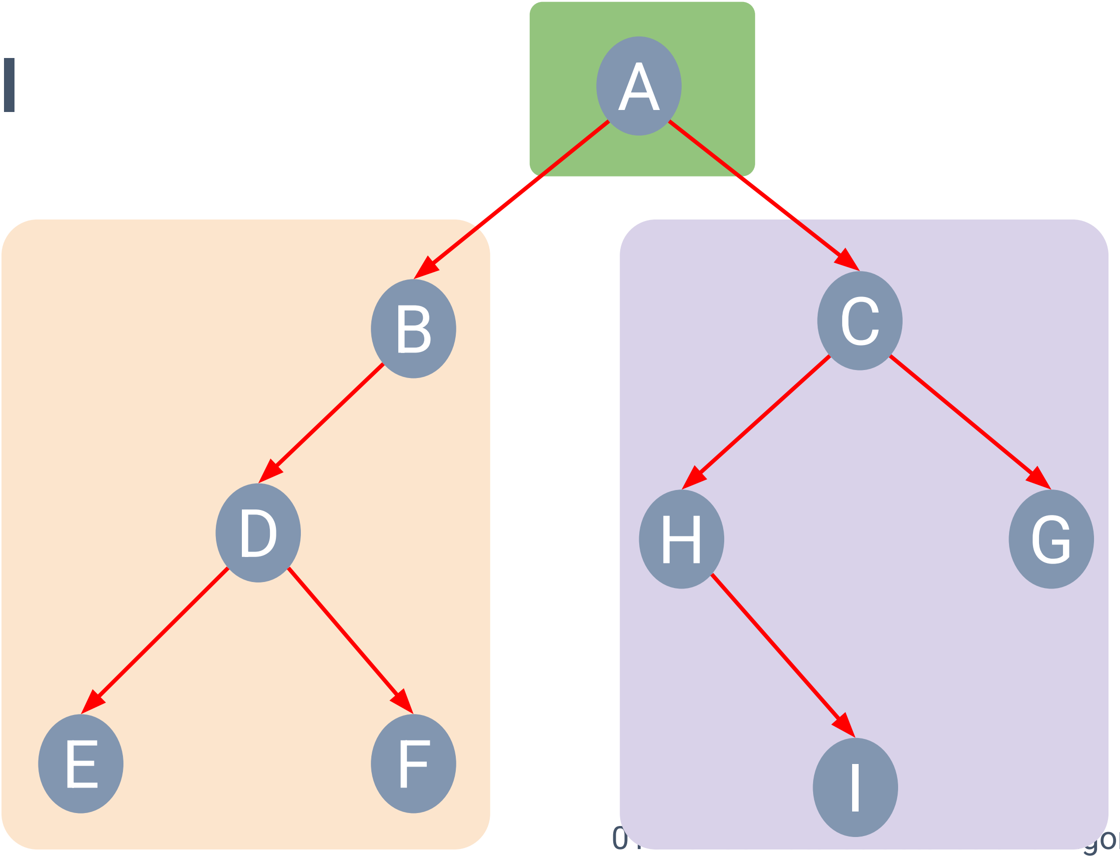
Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**

A



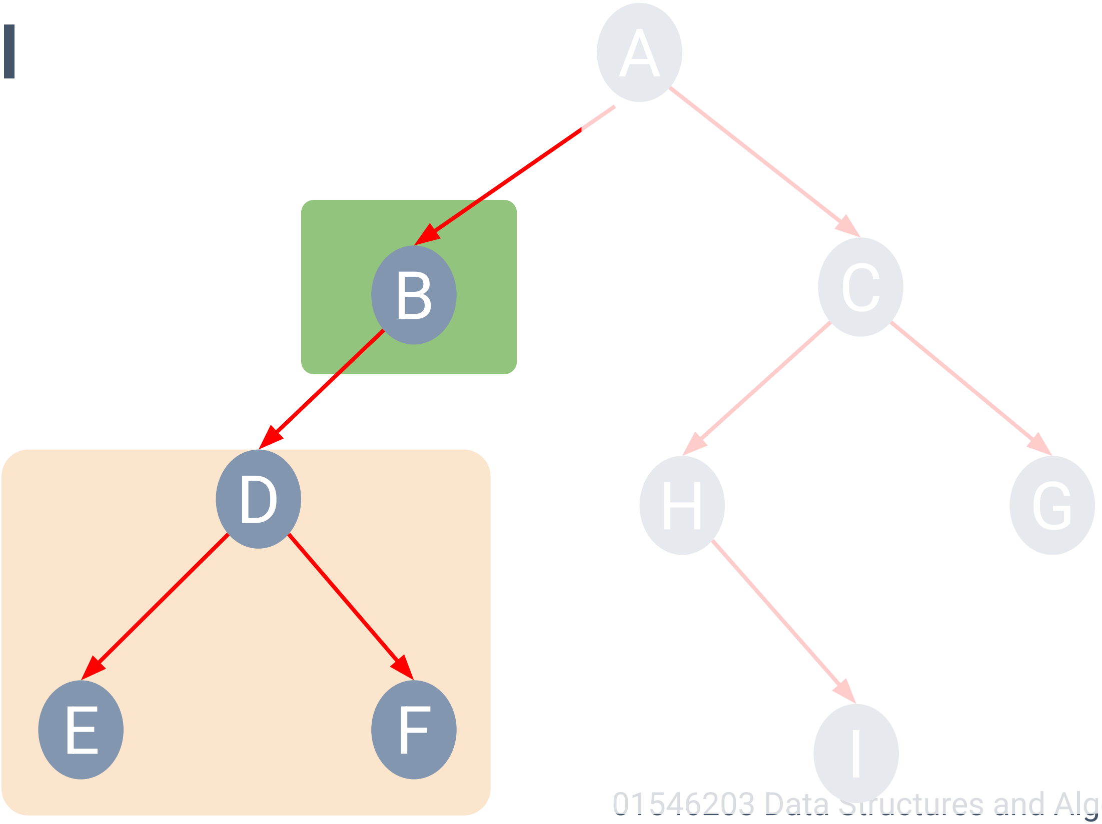
Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**

A B



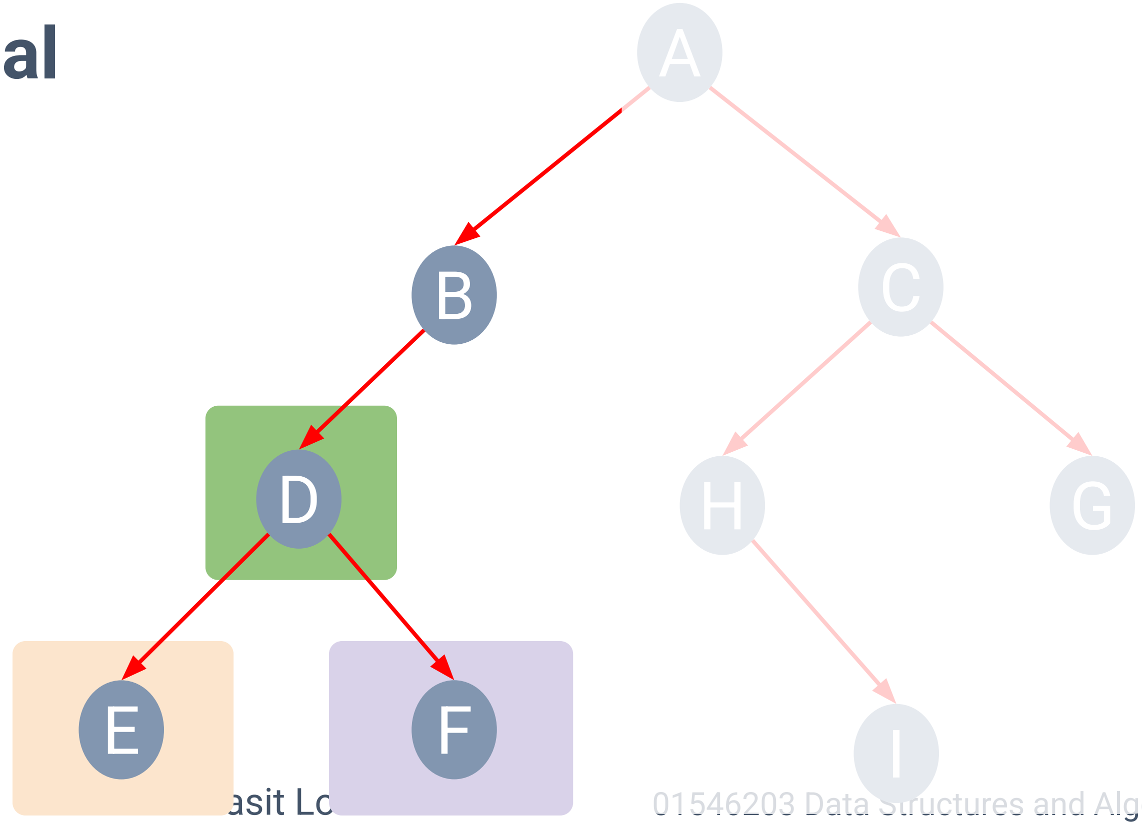
Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**

A B D E F



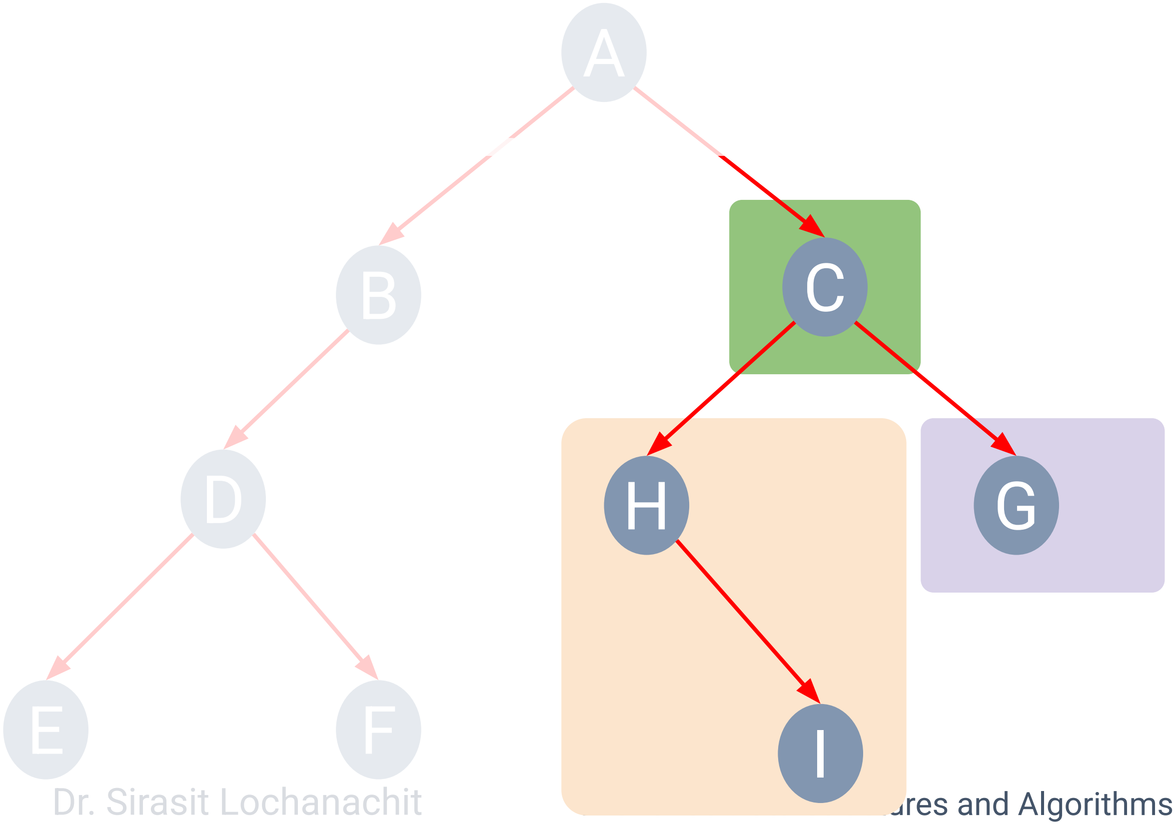
Preorder Traversal

- **Preorder Traversal**

  N L R

- **Visited nodes**

A B D E F



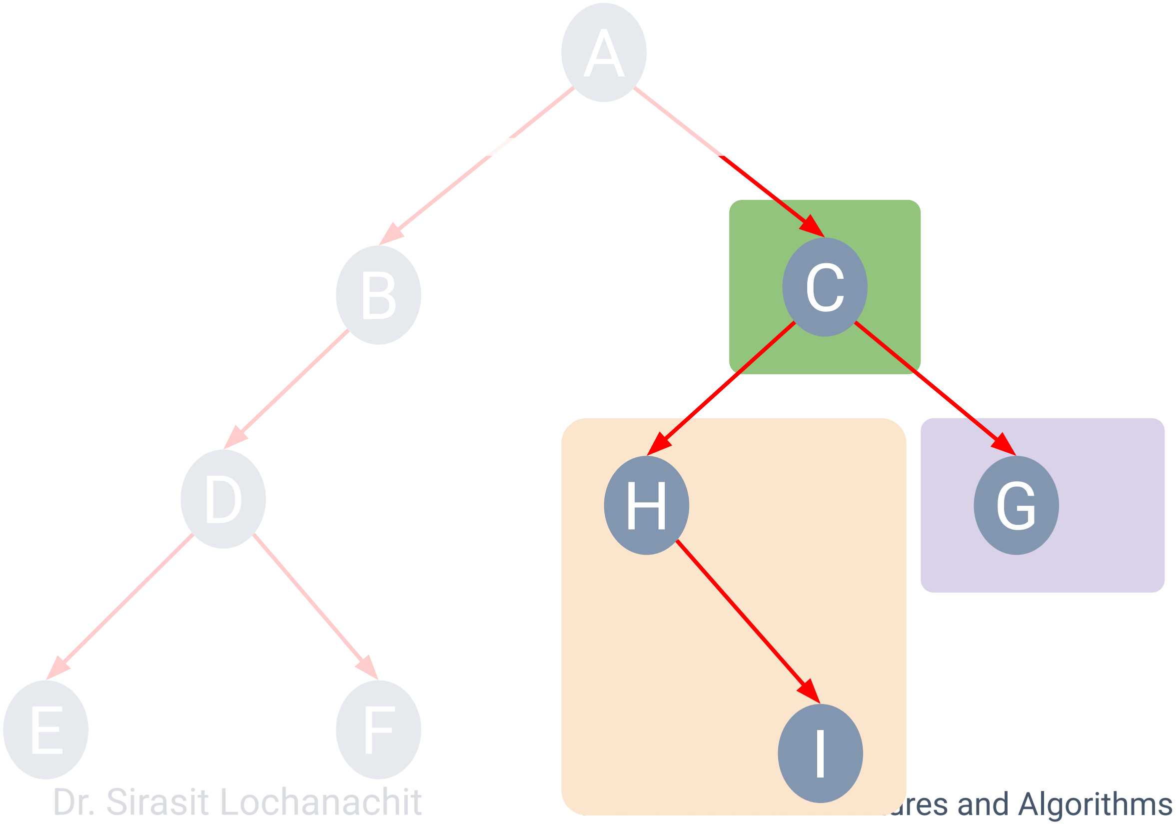
Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**

A B D E F C



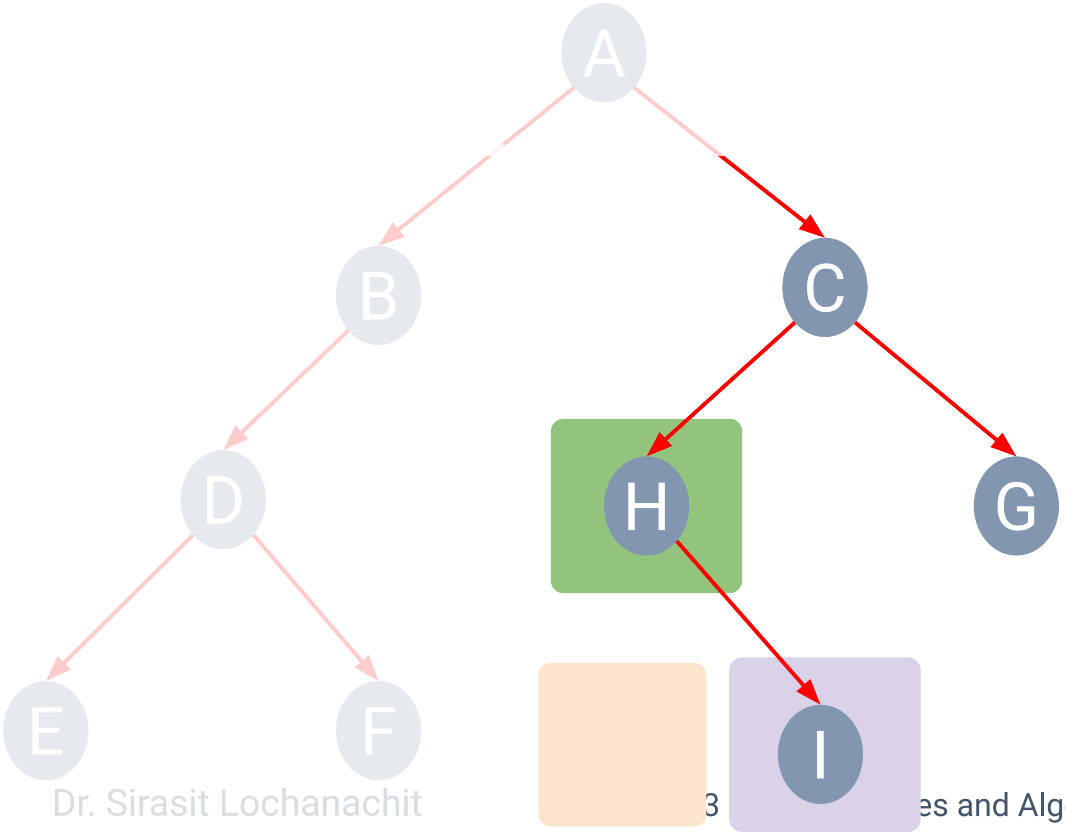
Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**

A B D E F
C H I



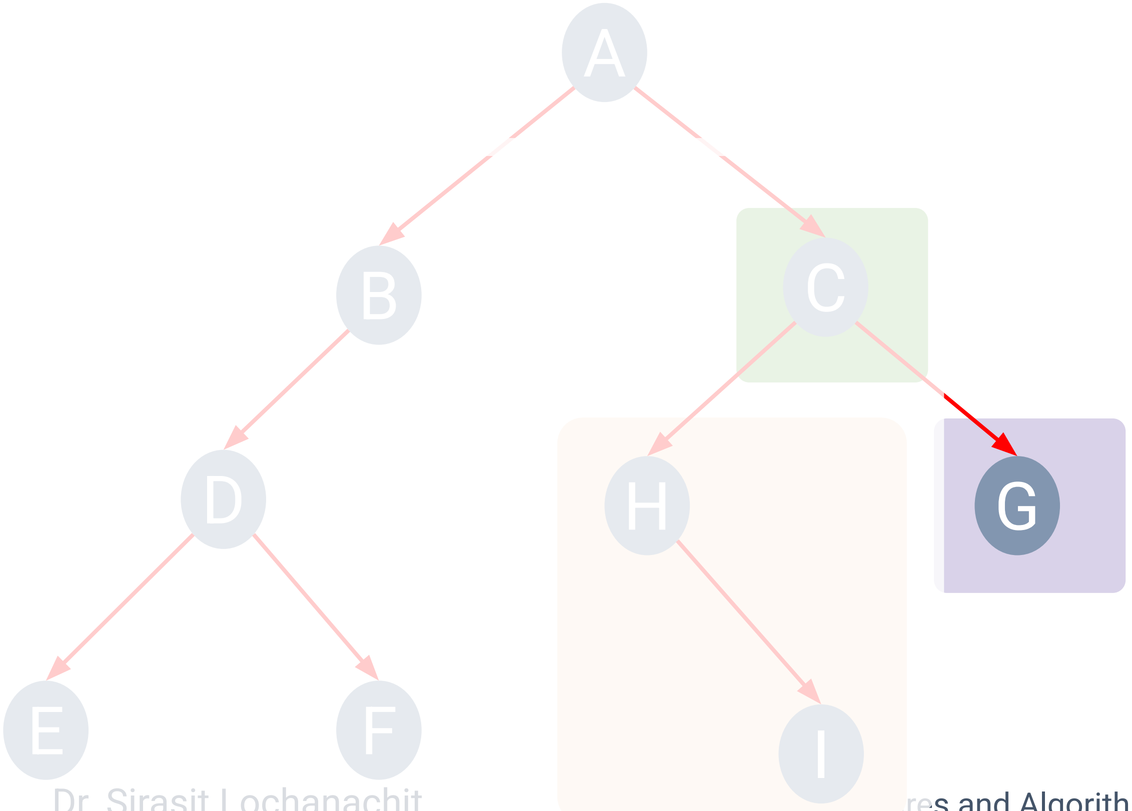
Preorder Traversal

- **Preorder Traversal**

N L R

- **Visited nodes**

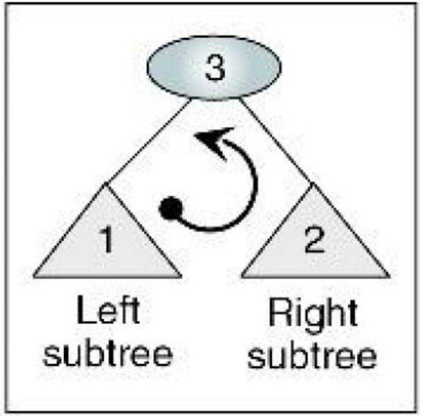
A B D E F
C H I G



Postorder Traversal

- Left subtree -> Right subtree -> root

```
Algorithm postOrder(root):
    if (root is not null):
        postOrder(leftSubTree)
        postOrder(rightSubtree)
        process(root)
    end if
end postOrder
```

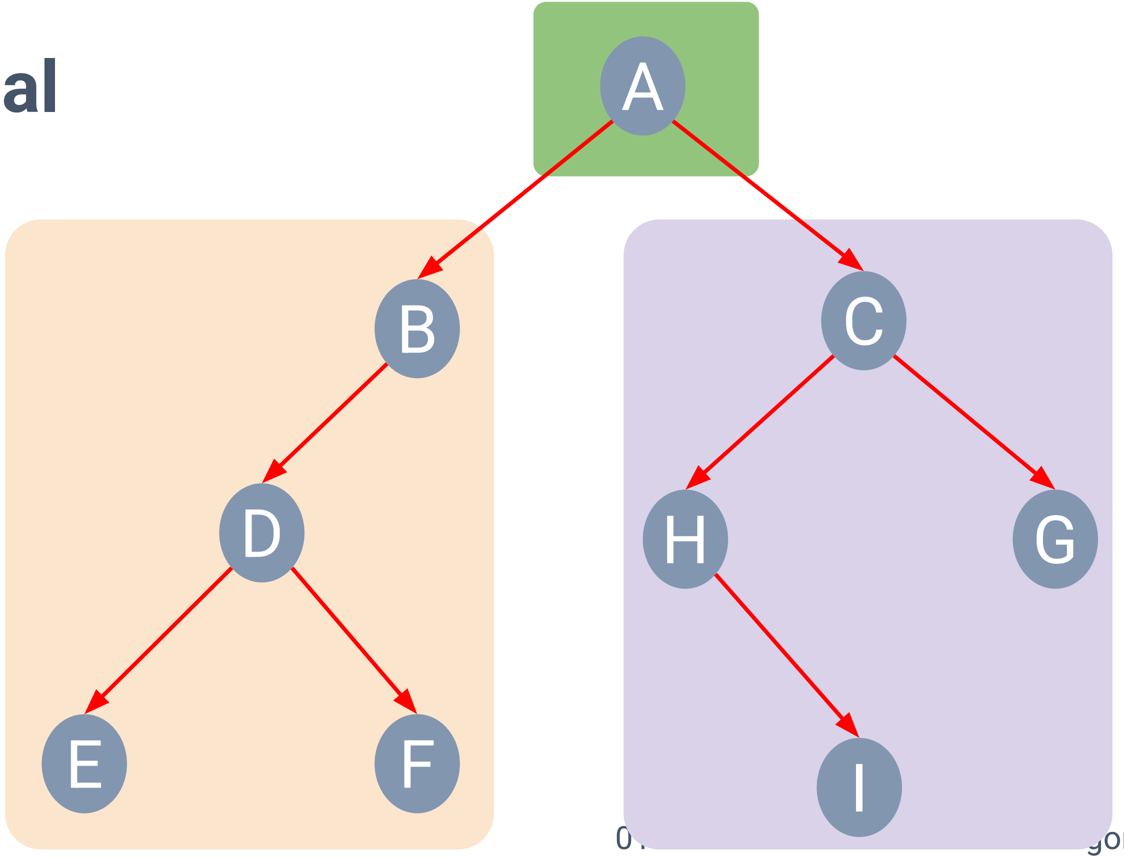


Postorder Traversal

- **Postorder Traversal**

  L R N

- **Visited nodes**

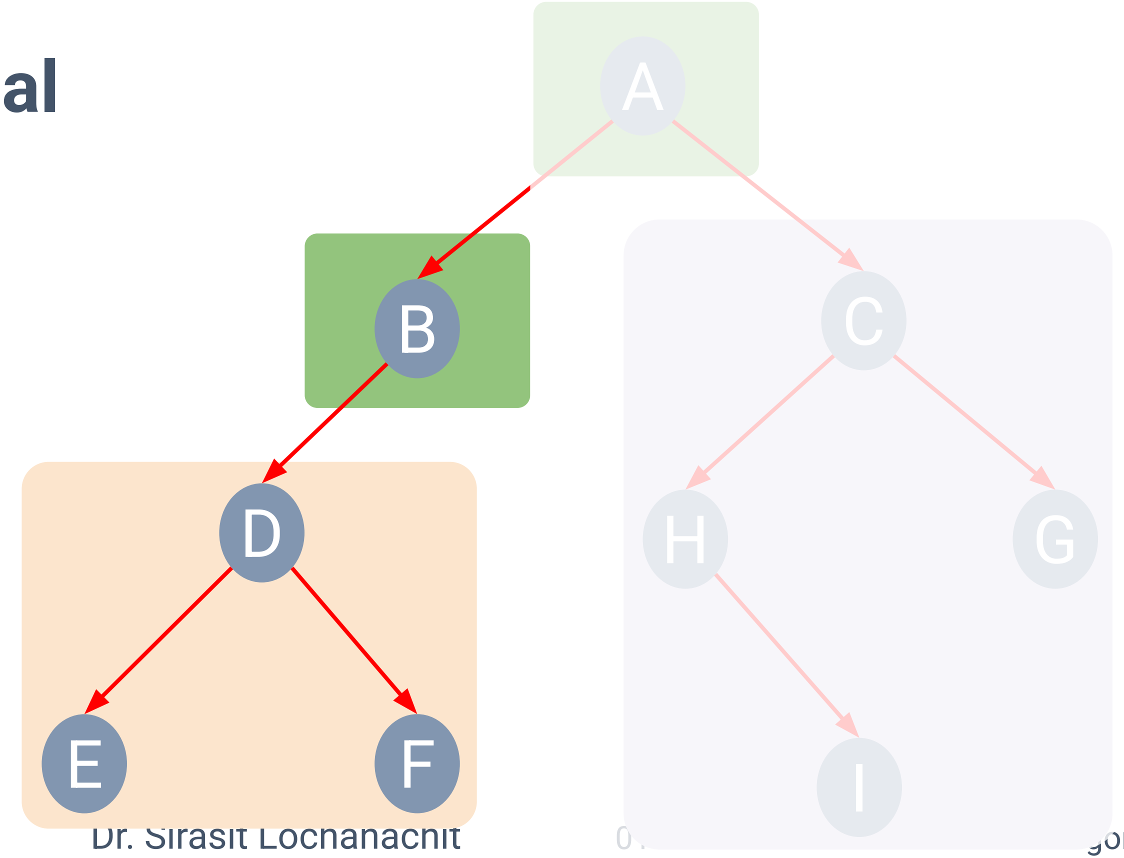


Postorder Traversal

- **Postorder Traversal**

L R N

- **Visited nodes**



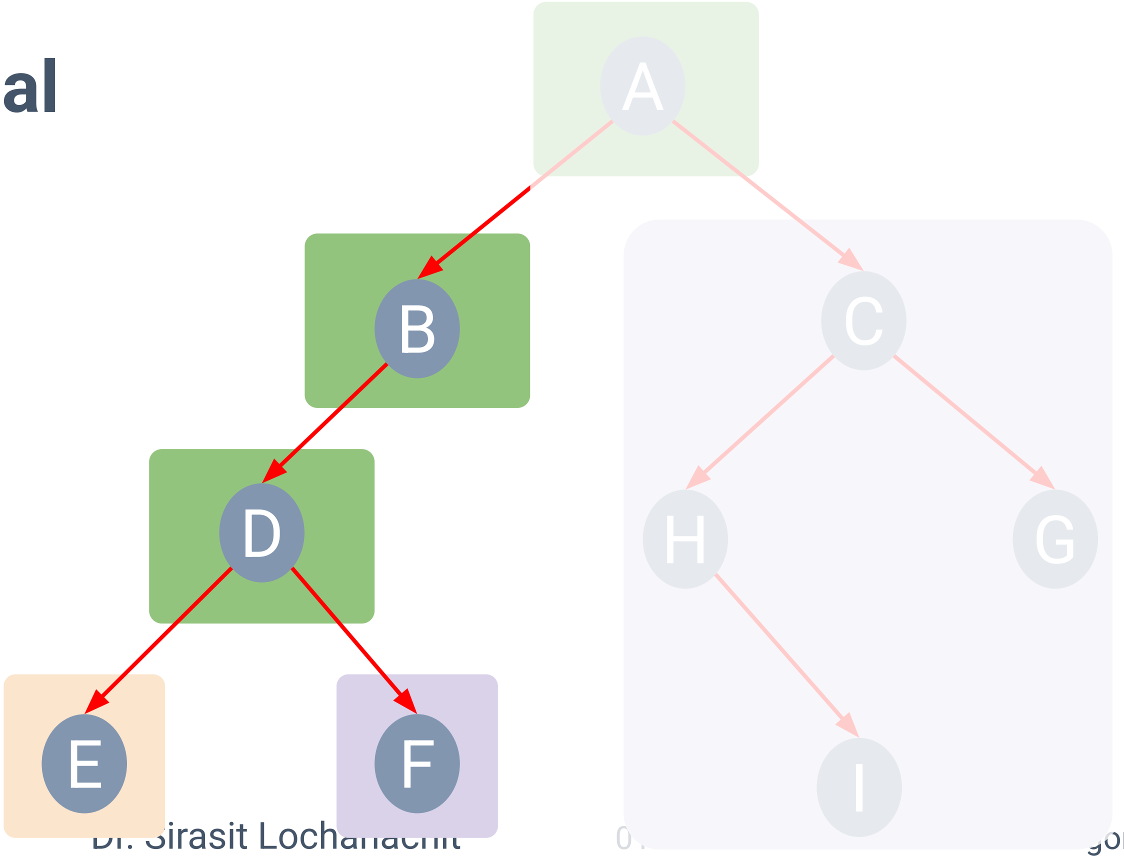
Postorder Traversal

- **Postorder Traversal**

L R N

- **Visited nodes**

E F



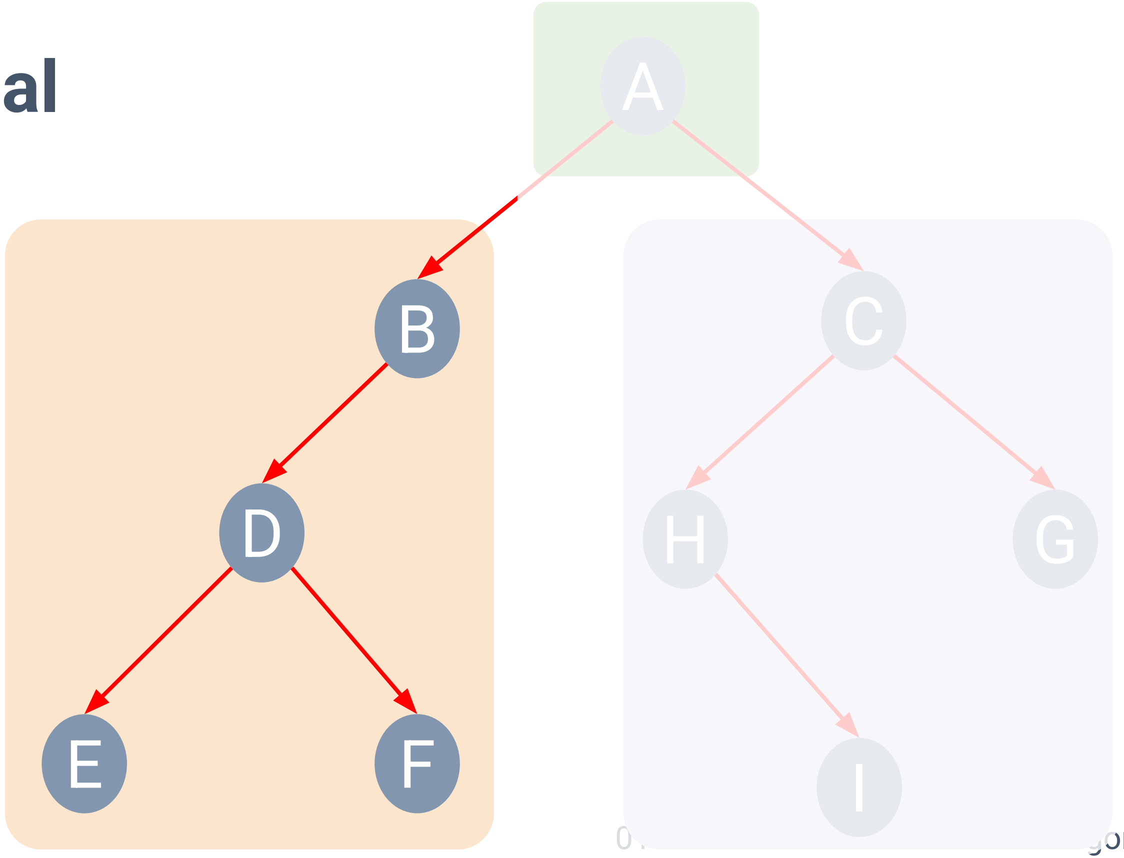
Postorder Traversal

- **Postorder Traversal**

  L R N

- **Visited nodes**

E F D B



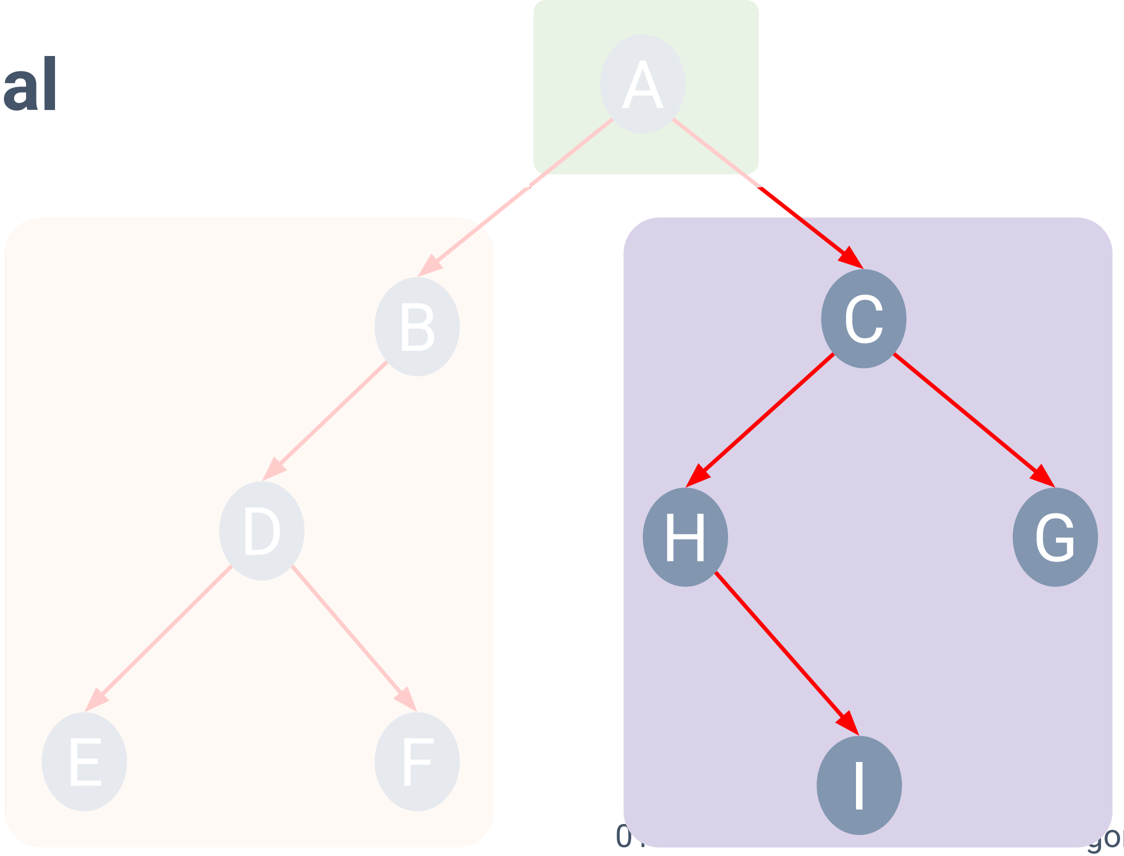
Postorder Traversal

- **Postorder Traversal**

L R N

- **Visited nodes**

E F D B
I H G C



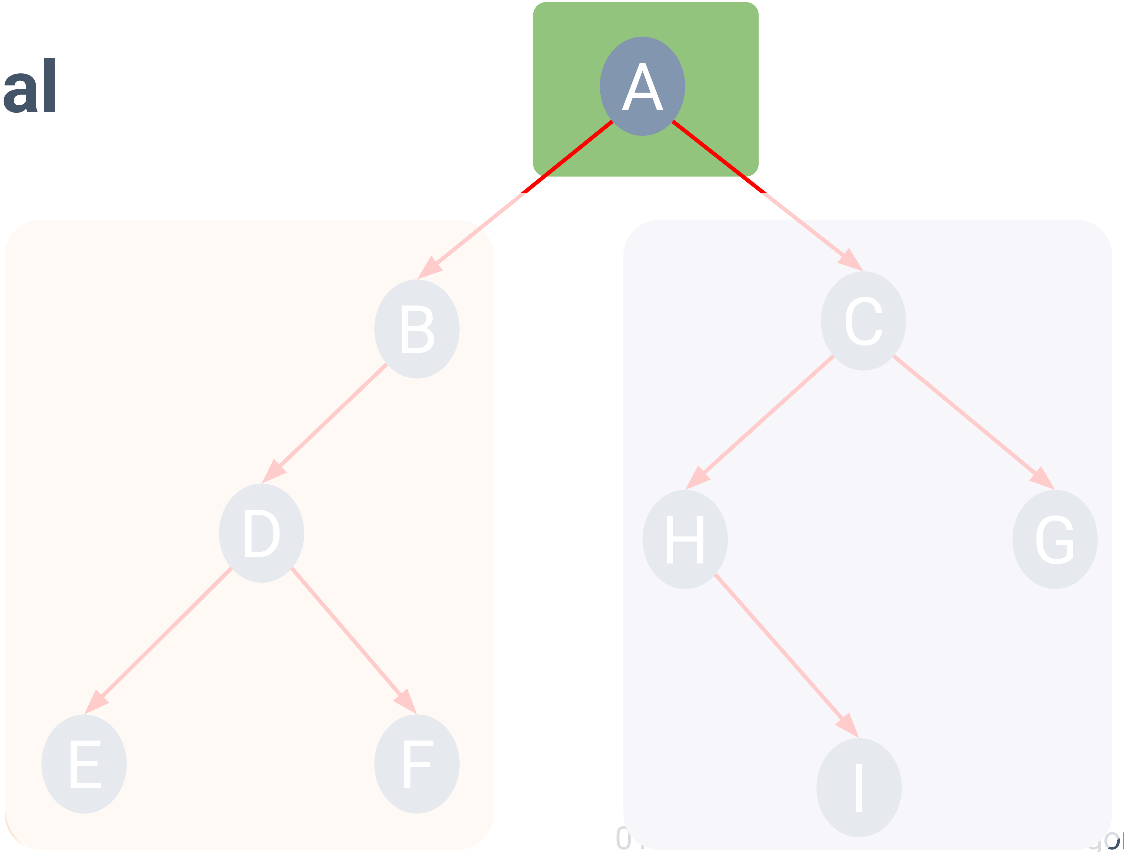
Postorder Traversal

- **Postorder Traversal**

L R N

- **Visited nodes**

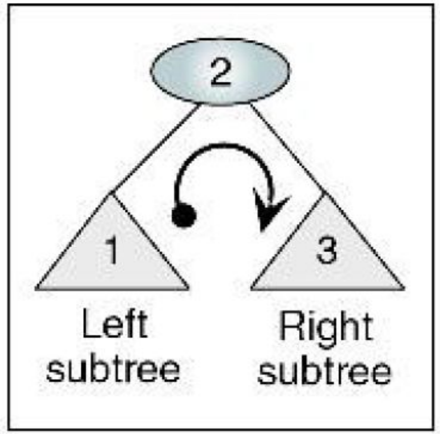
E F D B
I H G C
A



Inorder Traversal

- Left subtree -> root -> Right subtree

```
Algorithm inOrder(root):
    if (root is not null):
        inOrder(leftSubTree)
        process(root)
        inOrder(rightSubtree)
    end if
end inOrder
```

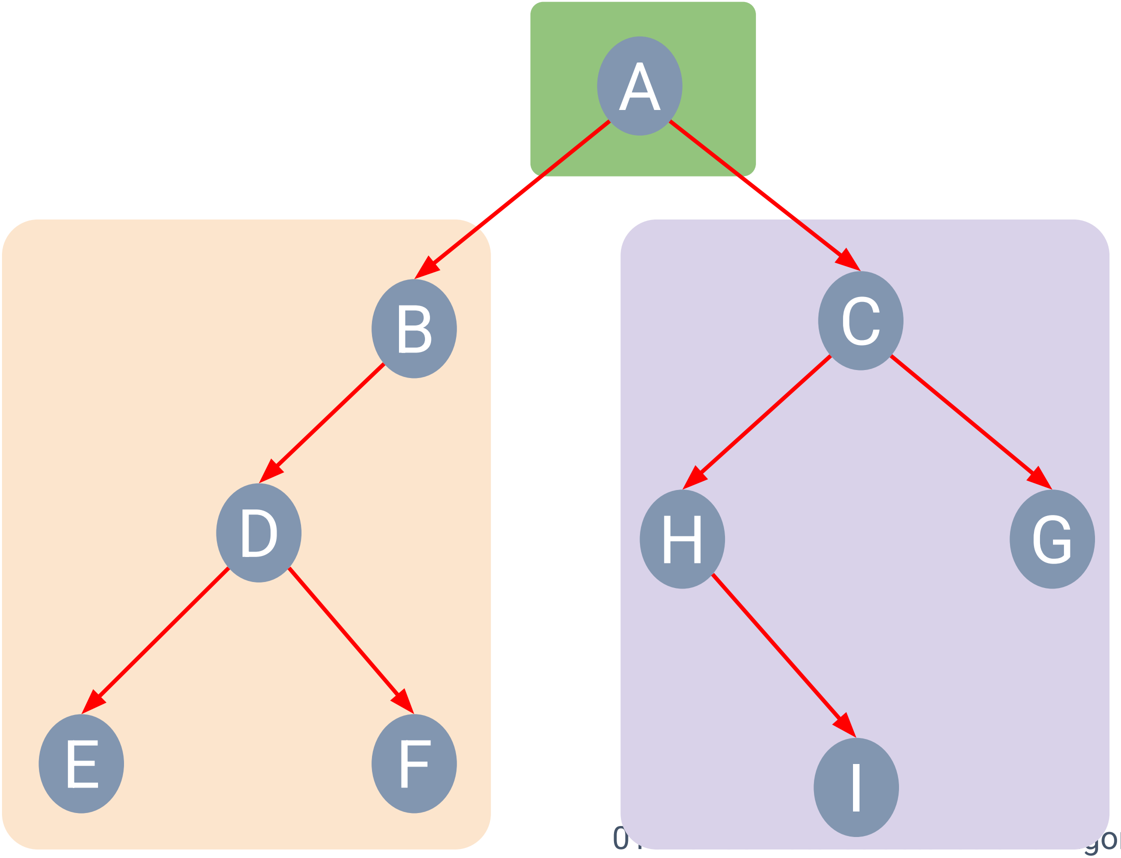


Inorder Traversal

- **Inorder Traversal**

  L N R

- **Visited nodes**



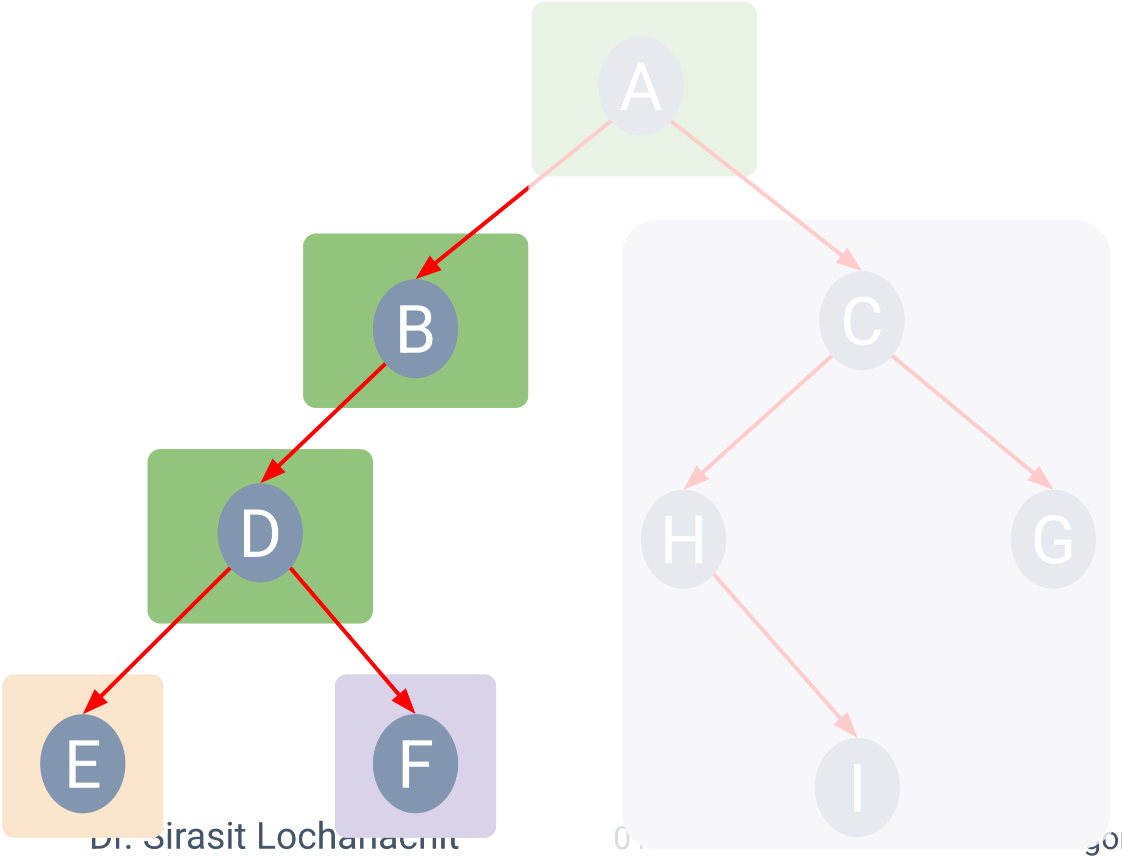
Inorder Traversal

- **Inorder Traversal**

  L N R

- **Visited nodes**

  E D F



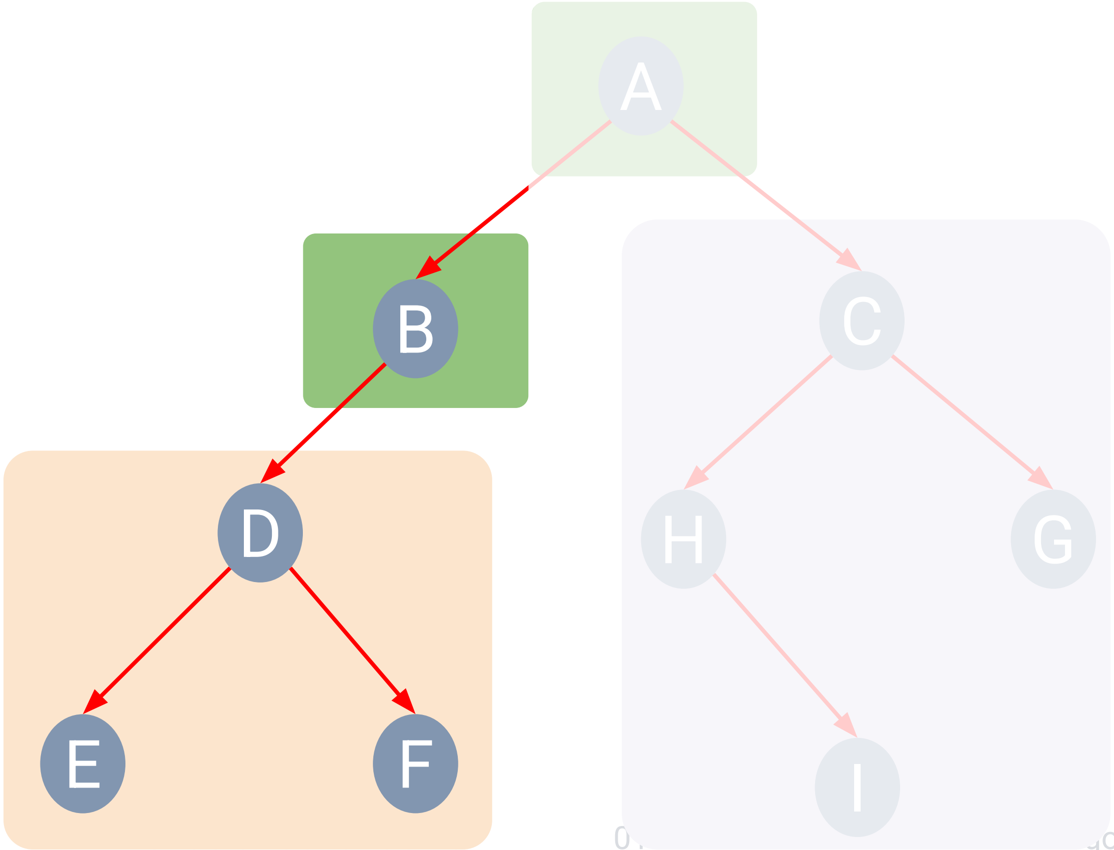
Inorder Traversal

- **Inorder Traversal**

  L N R

- **Visited nodes**

  E D F B



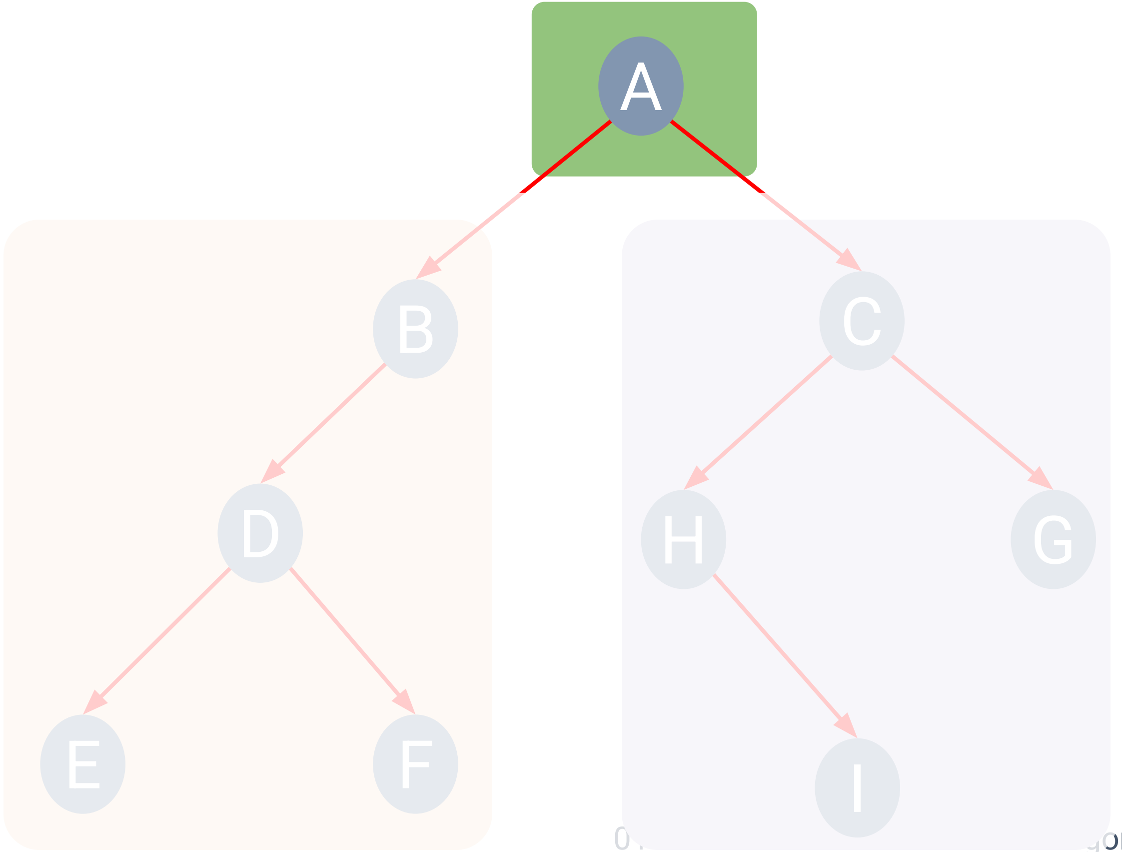
Inorder Traversal

- **Inorder Traversal**

L N R

- **Visited nodes**

E D F B A



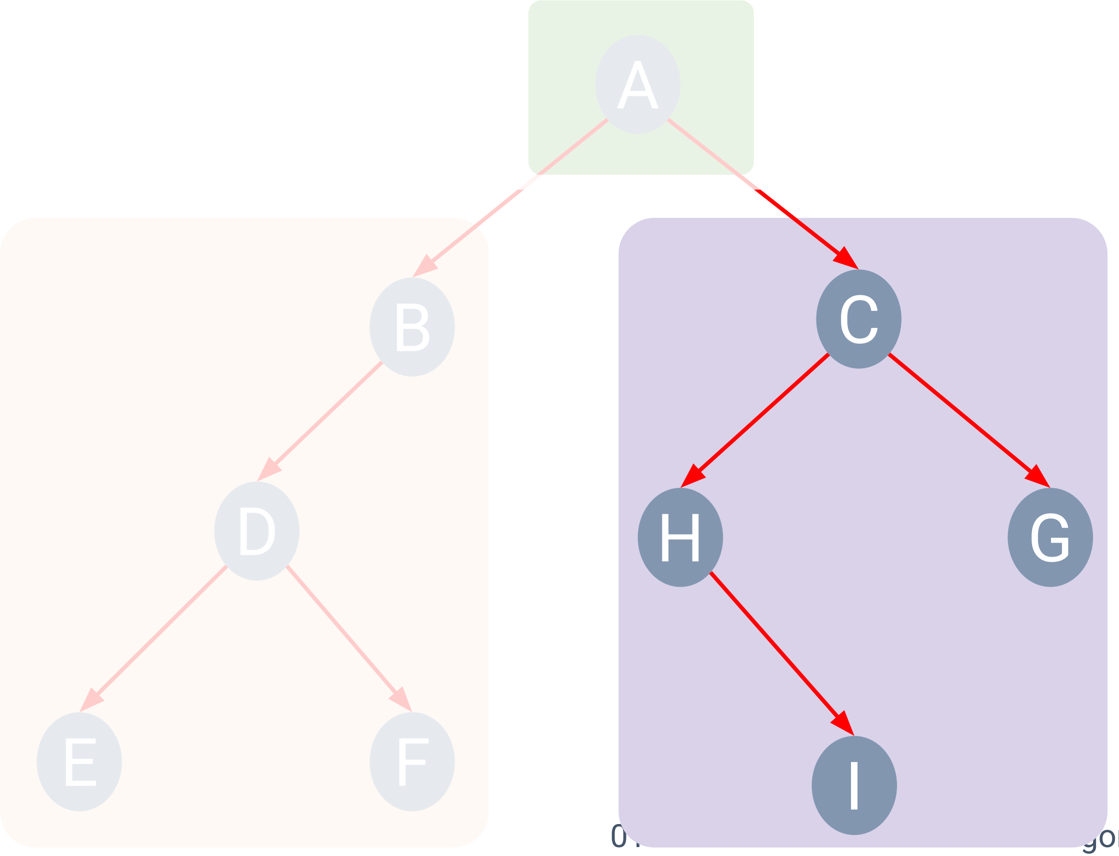
Inorder Traversal

- **Inorder Traversal**

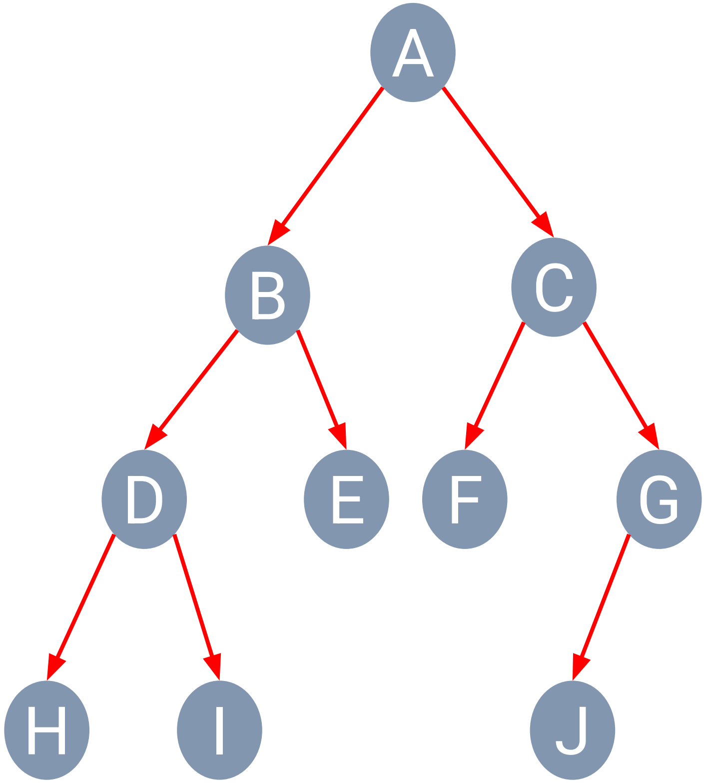
  L N R

- **Visited nodes**

  E D F B A
  H I C G

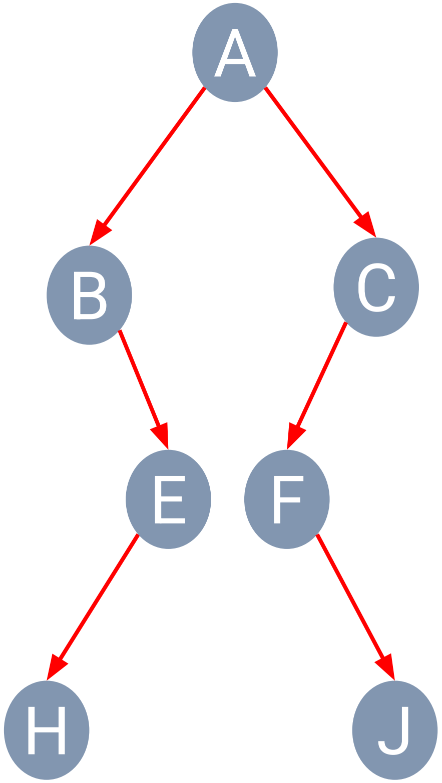


Depth-first Traversal Exercise 1



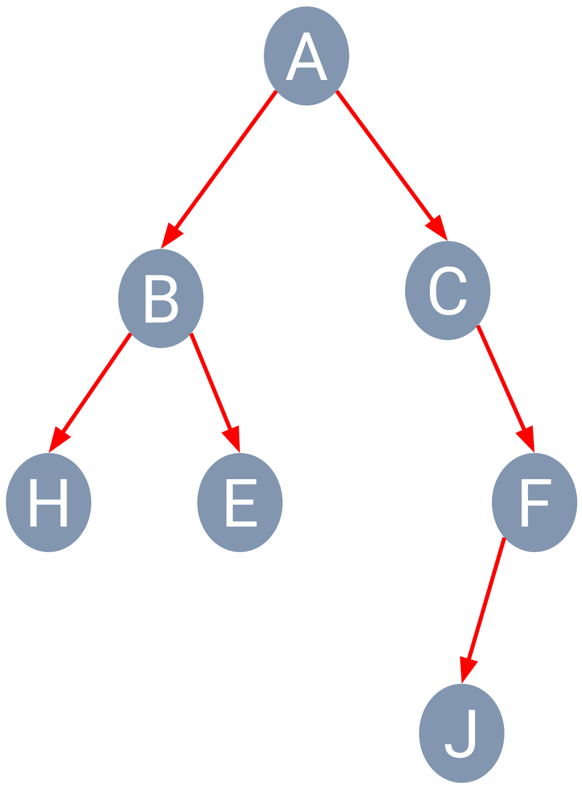
- **Preorder Traversal** N L R
- **Inorder Traversal** L N R
- **Postorder Traversal** L R N

Depth-first Traversal Exercise 2



- Preorder Traversal N L R
- Inorder Traversal L N R
- Postorder Traversal L R N

Depth-first Traversal Exercise 3



- **Pre**order Traversal N L R
- **In**order Traversal L N R
- **Post**order Traversal L R N

Quiz

1. Find root of Binary Tree from a given traversal path:
   a. Tree with postorder traversal: FCBDG
   b. Tree with preorder traversal: IBCDFEN
   c. Tree with inorder traversal: CBIDFGE

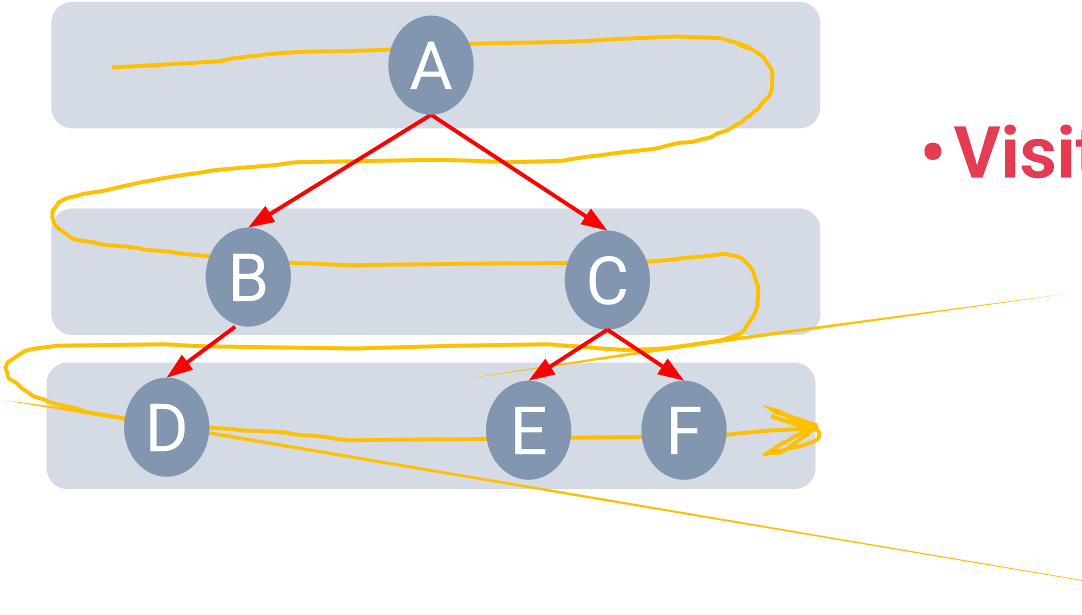
Quiz

2. Draw a Binary Tree from a given traversal path:

    a. Preorder: JCBADEFIGH

    b. Inorder: ABCEDFJGIH

Breadth-first Traversal



- Visited nodes

- Visit all the nodes at depth *d* before moving to depth *d*+1.